



T.C.

NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖRÜNTÜ İŞLEME DESTEKLİ OTONOM KAVŞAK
VE TRAFİK YÖNETİM SİSTEMİ

SEDAT EROĞLU

210610044

BERK ALTINÖZ

220610017

BİTİRME TEZİ

HAKAN AKTAŞ

NİĞDE 2026



T.C.
NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

GÖRÜNTÜ İŞLEME DESTEKLİ OTONOM KAVŞAK
VE TRAFİK YÖNETİM SİSTEMİ

SEDAT EROĞLU

210610044

BERK ALTINÖZ

220610017

BİTİRME TEZİ

HAKAN AKTAŞ

NİĞDE 2026

NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BİTİRME TEZİ KABUL VE ONAY BELGESİ

Bölümümüz 210610044 numaralı öğrencisi SEDAT EROĞLU'nun Görüntü İşleme Destekli Otonom Kavşak ve Trafik Yönetim Sistemi başlıklı Bitirme Tezi çalışması aşağıdaki jüri üyeleri tarafından Bilgisayar Mühendisliği Bölümü'nde Bitirme Tezi olarak Oy Birliği/Oy Çokluğu ile kabul edilmiştir.

Danışman : Dr. Öğr. Üyesi Hakan AKTAŞ

Üye : Dr. Öğr. Üyesi Yeşim DOKUZ

Üye : Doç. Dr. Ahmet Şakir DOKUZ

Üye : Doç. Dr. Erkan ÇALIŞKAN

Üye : Dr. Öğr. Üyesi Ahmet Can DENİZ

Tezin savunulduğu Tarih: .../.../...

Bitirme Tezi dersi kapsamında yapılan bu çalışma, ilgili jüriler tarafından değerlendirme sonucunda b Mühendisliği Bölümü'nde Bitirme Tezi çalışması olarak kabul edilmiştir.

.../.../...

Bölüm Başkanı

Doç. Dr. Ahmet Şakir DOKUZ

NİĞDE ÖMER HALİSDEMİR ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ
BİTİRME TEZİ KABUL VE ONAY BELGESİ

Bölümümüz 210610044 numaralı öğrencisi BERK ALTINÖZ’ün Görüntü İşleme Destekli Otonom Kavşak ve Trafik Yönetim Sistemi başlıklı Bitirme Tezi çalışması aşağıdaki jüri üyeleri tarafından Bilgisayar Mühendisliği Bölümü’nde Bitirme Tezi olarak Oy Birliği/Oy Çokluğu ile kabul edilmiştir.

Danışman : Dr. Öğr. Üyesi Hakan AKTAŞ

Üye : Dr. Öğr. Üyesi Yeşim DOKUZ

Üye : Doç. Dr. Ahmet Şakir DOKUZ

Üye : Doç. Dr. Erkan ÇALIŞKAN

Üye : Dr. Öğr. Üyesi Ahmet Can DENİZ

Tezin savunulduğu Tarih: .../.../...

Bitirme Tezi dersi kapsamında yapılan bu çalışma, ilgili jüriler tarafından değerlendirme sonucunda b Mühendisliği Bölümü’nde Bitirme Tezi çalışması olarak kabul edilmiştir.

.../.../...

Bölüm Başkanı

Doç. Dr. Ahmet Şakir DOKUZ

DOĞRULUK BEYANI

Bitirme tezi olarak sunduğum bu çalışmayı tüm akademik kurallara ve Niğde Ömer Halisdemir Üniversitesi Yayın Etiği Komisyonu Yönergesi'ne uygun olarak gerçekleştirdiğimi ve sunduğumu; bu kurallar ve ilkelere aykırı hiçbir yol ve yardıma başvurmaksızın bizzat hazırladığımı beyan ederim.

Tezimle ilgili yaptığım beyana aykırı bir durum saptanırsa ortaya çıkacak tüm ahlaki ve hukuki sonuçlara katlanacağımı bildiririm.

BERK ALTINÖZ

DOĞRULUK BEYANI

Bitirme tezi olarak sunduđum bu alıřmayı tm akademik kurallara ve Niđde mer Halisdemir niversitesi Yayın Etiđi Komisyonu Ynergesi'ne uygun olarak gerekleřtirdiđimi ve sunduđumu; bu kurallar ve ilkelere aykırı hibir yol ve yardıma bařvurmaksızın bizzat hazırladıđımı beyan ederim.

Tezimle ilgili yaptıđım beyana aykırı bir durum saptanırsa ortaya ıkacak tm ahlaki ve hukuki sonulara katlanacađımı bildiririm.

SEDAT EROĐLU

ÖZET

Lisans Tezi

GÖRÜNTÜ İŞLEME DESTEKLİ OTONOM KAVŞAK

VE TRAFİK YÖNETİM SİSTEMİ

SEDAT EROĞLU - BERK ALTINÖZ

Niğde Ömer Halisdemir Üniversitesi
Mühendislik Fakültesi
Bilgisayar Mühendisliği Bölümü

Danışman: Dr. Öğr. Üyesi HAKAN AKTAŞ

June 2026

Günümüzde hızla artan araç sayısı, kentiçi ulaşımında ciddi trafik sıkışıklıklarına yol açmaktadır. Geleneksel sabit zamanlı trafik ışıklarının bu dinamik yoğunluğa ayak uyduramadığı görülmektedir. Bu sorundan yola çıkılarak hazırlanan bu tez çalışmasında, görüntü işleme ve yapay zeka teknolojileri kullanılarak otonom bir akıllı kavşak yönetim sistemi tasarlanmıştır. 3 farklı yönün kamera görüntüleri anlık olarak işlenerek, kavşaktaki gerçek araç yoğunluğuna göre karar veren bir altyapı kurgulanmıştır.

Sistemde araç tespiti için derin öğrenme modellerinden YOLO algoritması tercih edilmiştir. Sistemin canlı kamera yayınlarında yüksek performansla çalışabilmesi için görüntü çözünürlükleri optimize edilmiş ve sadece araç sınıfını tespit edecek özel filtrelemeler uygulanmıştır. Kameradan alınan anlık araç sayıları ve kuyrukta bekleme süreleri, adil geçiş hakkı sağlamak üzere geliştirilen özel bir karar algoritmasına aktarılmaktadır. Bu algoritma, hangi yöne ne kadar süre yeşil ışık yanacağını dinamik olarak hesaplamaktadır. Arka plan yazılımı ile arayüz arasındaki anlık veri haberleşmesi ise Flask-SocketIO mimarisi kullanılarak sağlanmıştır.

Çalışmanın bir diğer önemli adımı ise sistemin uzaktan izlenip yönetilebilmesi için tasarlanan SCADA tabanlı web arayüzüdür. React ile geliştirilen bu kontrol paneli üzerinden kameraların anlık durumu, sistem metrikleri ve ışık süreleri canlı olarak takip edilebilmektedir. Yapılan testler, geliştirilen dinamik sistemin klasik yöntemlere kıyasla ortalama bekleme sürelerini düşürdüğünü ve kavşak verimliliğini artırdığını göstermektedir.

Anahtar Kelimeler: Akıllı Kavşak, YOLO, Görüntü İşleme, Trafik Optimizasyonu.

SUMMARY

Undergraduate Thesis

GÖRÜNTÜ İŞLEME DESTEKLİ OTONOM KAVŞAK

VE TRAFİK YÖNETİM SİSTEMİ

SEDAT EROĞLU - BERK ALTINÖZ

Nigde Ömer Halisdemir University
Faculty of Engineering
Department of Computer Engineering

Advisor: Asst. Prof. Dr. Hakan AKTAŞ

June 2026

The rapidly increasing number of vehicles today is causing serious traffic congestion in urban transportation. It appears that traditional fixed-timed traffic lights cannot keep up with this dynamic traffic density. In this thesis prepared based on this problem, an autonomous intelligent intersection management system has been designed using image processing and artificial intelligence technologies. An infrastructure has been designed that processes camera images from 3 different directions in real time and makes decisions based on the actual vehicle density at the intersection.

For vehicle detection in the system, the YOLO algorithm, a deep learning model, was chosen. In order for the system to work with high performance in live camera broadcasts, image resolutions have been optimized and special filters have been applied to detect only the vehicle class. The instantaneous vehicle numbers and queue waiting times taken from the camera are transferred to a special decision algorithm developed to ensure fair transit rights. This algorithm dynamically calculates how long the green light will stay on in each direction. The real-time data communication between the background software and the interface is achieved using the Flask-SocketIO architecture.

Another important step in the project is the SCADA-based web interface designed for remote monitoring and management of the system. Through this control panel developed with React, the current status of cameras, system metrics, and light timers can be monitored live. The tests carried out show that the dynamic system developed reduces the average waiting times compared to the classical methods and increases the efficiency of the intersection.

Keywords: Smart Intersection, YOLO, Image Processing, Traffic Optimization.

İÇİNDEKİLER

ÖZET	vi
SUMMARY	vii
KISALTMALAR DİZİNİ.....	xi
ŞEKİLLER DİZİNİ.....	xii
ÖNSÖZ	xiii
BÖLÜM 1	1
1.1. Projeye Genel Bakış	1
1.2. Anomali Tespiti İhtiyacı.....	1
1.3. Tezin Amacı ve Kapsamı	1
1.4. Katkılar ve Yenilikçi Yönler	2
BÖLÜM 2	3
2.1. Yazılım Teknolojileri ve Geliştirme Ortamı	3
2.1.1. Görüntü İşleme ve Yapay Zeka (YOLO ve OpenCV)	3
2.1.2. Arka Plan Mimarisi ve Haberleşme (Python, Flask ve SocketIO).....	4
2.1.3. Kullanıcı Arayüzü (React Tabanlı SCADA).....	4
2.2. Donanım Mimarisi ve Prototip Katmanı	5
2.2.1. Merkezi İşlem Birimi (Ana Sunucu)	5
2.2.2. Saha Kontrol Birimi (Mikrodenetleyici Entegrasyonu)	5
2.2.3. Endüstriyel Tip Sinyalizasyon Bileşenleri ve Sayaçlar.....	6
2.3. Backend Teknolojileri	6
2.3.1. Flask ve Flask-SocketIO ile Gerçek Zamanlı Haberleşme.....	7
2.3.2. YOLOv8 ve PyTorch ile Gerçek Zamanlı Araç Tespiti.....	8
2.3.4. Adaptif Trafik Kontrol Algoritması (Karar Mekanizması)	11
2.3.5. MerkezTakip Sınıfı ile Hareket Analizi	12
2.3.6. Video İşleme Ana Döngüsü ve Veri İletimi	12
2.4. Frontend Teknolojileri.....	13
2.4.1. Kurumsal Kimlik ve Bilgilendirme Ekranları	14
2.4.2. Gerçek Zamanlı Kamera Paneli ve Optimizasyon	15
2.4.3. Sistem Metrikleri ve Operatör Kontrol Modülü	16

BÖLÜM 3	18
3.1. Veri Edinme ve Ön İşleme Katmanı (OpenCV Edge Interface)	19
3.2. Çıkarım ve Çekirdek Analiz Katmanı (YOLOv8 Core & PyTorch).....	19
3.3. Adaptif Karar ve Optimizasyon Katmanı (Control Logic).....	19
3.4. İletişim ve Dağıtım Katmanı (Flask-SocketIO & UART Gateway)	19
3.5. Sunum ve Kontrol Katmanı (React SCADA)	19
BÖLÜM 4	21
4.1. Hareketsiz Araç ve Kaza Algılama Mantığı.....	21
4.2. Acil Durum Araçlarının Tanınması ve Önceliklendirme	21
4.3. Yanlış Alarmların Filtrelenmesi ve Kararlılık.....	22
BÖLÜM 5	23
TESTLER VE SONUÇLAR.....	23
5.1 Araç Yoğunluğu Tespit Testleri	23
5.2 Trafik Işığı Sürelerinin Dinamik Olarak Ayarlanması.....	23
5.3 Arduino ve Seri Haberleşme Testleri	23
5.4 Sistemin Manuel Modda Çalıştırılması	24
5.5 Genel Değerlendirme.....	24
BÖLÜM 6	25
SONUÇ VE GELECEK ÖNERİLER.....	25
6.1 Projenin Başlıca Kazanımları	25
6.2 Gelecek Çalışmalar İçin Öneriler	25
6.2.1 Acil Durum Araçlarına Öncelik Verilmesi.....	25
6.2.2 Gelişmiş Araç Sınıflandırma Sistemleri.....	25
6.2.3 Gerçek Trafik Kameraları ile Entegrasyon.....	26
6.2.4 Merkezi Trafik Yönetim Altyapısı	26
6.2.5 Mobil ve Web Tabanlı Yönetim Sistemleri.....	26
6.3 Sistemin Geliştirilebilir Yönleri	26
6.3.1 Kamera ve Algılama Performansının Artırılması.....	26
6.3.2 Büyük Ölçekli Kavşak Yapılarının Desteklenmesi.....	26
6.3.3 Bulut Tabanlı Veri Analizi	26
6.3.4 Akıllı Şehir Sistemleri ile Entegrasyon	26

KAYNAKÇA.....	27
EK-1 Risk Yönetim Tablosu.....	28
RİSK YÖNETİMİ TABLOSU.....	28
EK-2 Standartlar ve Kısıtlar Formu	29
STANDARTLAR VE KISITLAR FORMU	29
EK-3 İş-Zaman Çizelgesi.....	31
İŞ-ZAMAN ÇİZELGESİ	31
ÖZGEÇMİŞ	32
ÖZGEÇMİŞ	33

KISALTMALAR DİZİNİ

AI : Artificial Intelligence (Yapay Zeka)

API : Application Programming Interface (Uygulama Geliştirme Arayüzü)

CPU : Central Processing Unit (Merkezi İşlem Birimi)

FPS : Frames Per Second (Saniyedeki Kare Sayısı)

GPU : Graphics Processing Unit (Grafik İşlem Birimi)

HTTP : HyperText Transfer Protocol (Üst Metin Aktarma Protokolü)

OpenCV : Open Source Computer Vision (Açık Kaynaklı Görüntü İşleme Kütüphanesi)

REST : Representational State Transfer (Temsili Durum Aktarımı - Web Servis Mimarisi)

RTU : Remote Terminal Unit (Uzak Terminal Birimi - Saha Kontrolcüsü)

SCADA : Supervisory Control and Data Acquisition (Merkezi Denetim ve Veri Toplama Sistemi)

Socket.IO : (Gerçek Zamanlı ve Çift Yönlü Web Haberleşme Kütüphanesi)

UART : Universal Asynchronous Receiver-Transmitter (Evrensel Asenkron Alıcı-Verici)

YOLO : You Only Look Once (Gerçek Zamanlı Nesne Tespiti Algoritması)

ŞEKİLLER DİZİNİ

Şekil 2. 1: YOLO Algoritmasının Tek Aşamalı Nesne Tespit Mimarisi [1]	3
Şekil 2. 2: İstemci ve Sunucu Arasındaki Çift Yönlü Gerçek Zamanlı Veri Akışı.	4
Şekil 2. 3: Python Sunucu ve Arduino Arasındaki Veri İletişim Protokolü.	5
Şekil 2. 4: Backend'in Üç Aşamalı Diyagramı	6
Şekil 2. 5: Flask Uygulamasının Başlatılması Ve Temel Yapılandırması	7
Şekil 2. 6: İstemciden Gelen Otomatik Komutlar	7
Şekil 2. 7: Model Dosyasının Yüklenmesi.....	8
Şekil 2. 8: Videoların Akışını Oluşturulması	9
Şekil 2. 9: Nesnelerin İşlenmesi ve Boyut Filtrelemesi	9
Şekil 2. 10: Video Akışı Döngüsü Ve Hata Yönetimi	10
Şekil 2. 11: Karelerin Jpeg Yapılması Ve Metin Tabanlı Formata Çevirilmesi	11
Şekil 2. 12: Limitler Aşıldığında, Sarı Işıklı Güvenli Geçiş Başlatma Fonksiyonu .	11
Şekil 2. 13: Araç Takibi Ve Sayım Mekanizması.....	12
Şekil 2. 14: Karar Mekanizması Güncelleme	13
Şekil 2. 15: İşlenen Karelerin İstemciye Gönderilmesi	13
Şekil 2. 16: Kimlik Ve Bilgilendirme Ekranı	14
Şekil 2. 17: Hizmetler Ekranı.....	14
Şekil 2. 18: Hakkımızdanın İçeriğindeki Misyonumuz Ekranı.....	15
Şekil 2. 19: Kamera içeriği arayüzü	15
Şekil 2. 20: Akıllı Kontrol Merkezi Menüsü	16
Şekil 3. 1: Sistemin Mimarisinin Tasarımı	19

ÖNSÖZ

Projemin her aşamasında bana desteklerini esirgemeyen, deneyimini ve bilgisini benimle paylaşan ve bakış açımı genişleten Niğde Ömer Halisdemir Üniversitesinden değerli hocamız Hakan Aktaş'a ve Lisans eğitim hayatım boyunca her türlü desteği sağlayan saygıdeğer tüm hocalarımıza teşekkürü borç bilirim. Bu süreçte hiçbir maddi manevi desteğini esirgemeyen aileme ve arkadaşlarıma en içten dileklerle teşekkür ederim.

SEDAT EROĞLU - BERK ALTINÖZ

Niğde, Haziran 2026

BÖLÜM 1

GİRİŞ

1.1. Projeye Genel Bakış

Hızla artan şehirleşme ve araç kullanımı, ulaşım ağlarının taşıma kapasitelerini her geçen gün daha fazla zorlaştırmaktadır. Özellikle üç yollu kavşak noktaları, trafik akışının en çok kesintiye uğradığı ve darboğazların olduğu kritik düğüm noktalarıdır. Mevcut altyapılarda yaygın olarak kullanılan sabit zamanlı trafik ışığı sistemleri, günün farklı saatlerinde değişen trafik yoğunluklarına uyum sağlayamamaktadır. Bu tez çalışması, kavşaklardaki anlık araç yoğunluğunu derin öğrenme yöntemleriyle analiz ederek ışık sürelerini dinamik olarak ayarlayan ve tüm bu süreci web tabanlı bir SCADA arayüzüyle sistem yöneticisine sunan otonom bir akıllı kavşak yönetim sisteminin tasarlanmasını ve geliştirilmesini konu almaktadır.

1.2. Anomali Tespiti İhtiyacı

Trafik yönetimi yalnızca yeşil, kırmızı ve sarı ışık döngülerinin işletilmesinden ibaret değildir. Sahada her an kazalar, beklenmedik yol çalışmaları veya ani yığılmalar gibi standart akışı bozan olağandışı durumlar yaşanabilir. Geleneksel sistemler bu tür "anomalilere" karşı tamamen kördür; trafik kilitlense dahi boş yola yeşil ışık yakmaya devam ederler. Sistemde görüntü işlemeye dayalı bir altyapının tercih edilmesinin temel nedeni, kavşaktaki olağan akışın dışına çıkıldığı anda (örneğin bir yönün bekleme süresi sınırı aştığında) bu durumu bir anomali olarak tespit edip sistemin kendi kendine acil durum kararları alabilmesini sağlamaktır.

1.3. Tezin Amacı ve Kapsamı

Bu çalışmanın temel amacı, kavşakları bir sisteme göre karar verebilen bir yapıya dönüştürmektir. Donanım ve yazılım süreçlerinin birbirine karışmadan stabil çalışabilmesi için proje altyapısı beş temel katmana ayrılarak tasarlanmıştır:

a. Donanım Katmanı: Sistemin fiziksel dünyayla temas ettiği uç noktadır. Kavşağın farklı yönlerini gören kamera birimlerini ve sahadan gelen bu yüksek çözünürlüklü görüntüleri işleyecek olan grafik işlemci/sunucu (GPU/CPU) altyapısını kapsar.

b. Veri İletim Katmanı: Sahadaki kameralardan elde edilen görüntülerin ve sistem durum mesajlarının ana karar merkezine taşındığı iletişim ağıdır. Projede bu katman, düşük gecikme süresi sunan ve çift yönlü iletişime izin veren WebSocket protokolleri üzerinden yapılandırılmıştır.

c. Veri Akışı Katmanı: Aynı anda üç farklı kameradan saniyede onlarca kare (frame) akmaktadır. Bu katman; Flask altyapısı kullanılarak görüntülerin birbirine karışmadan,

eşzamanlı bir şekilde sıraya dizilmesini ve yapay zeka modeline gönderilmeye hazır hale getirilmesini sağlamaktadır.

d. Veri Analiz Katmanı: Sistemin karar verici beynidir. Veri akışı katmanından gelen kareler, derin öğrenme tabanlı YOLO algoritmasına sokulmaktadır. Burada görüntülerdeki araçlar sayılmakta ve elde edilen yoğunluk verileri algoritmayla işlenerek hangi yöne ne kadar süre geçiş hakkı verileceği hesaplanmaktadır.

e. Kullanıcı Bildirim Katmanı: Arka planda çalışan yapay zeka ve karar algoritmalarının elde ettiği sonuçların, bir sistem yöneticisi tarafından izlenebilmesi için tasarlanan SCADA arayüzüdür. React tabanlı bu katman; kameraların verilerini, sistem metriklerini ve ışık durumlarını görselleştirerek kullanıcıya sunar ve gerektiğinde sisteme manuel müdahale imkanı tanır.

1.4. Katkılar ve Yenilikçi Yönler

Bu tez çalışmasının literatüre ve pratik saha uygulamalarına sunduğu en büyük katkı; Sabit süreli veya kısıtlı sensörlü geleneksel sistemlerin aksine; her aracı tek tek sayan ve gerektiğinde doğrudan müdahale edilebilen hibrid (yarı-otonom) bir yaklaşım ortaya konmuştur. Ayrıca YOLO algoritmasının, model boyutu ve hız optimizasyonlarıyla günlük bilgisayar donanımlarında bile anlık trafik analizi yapabilecek seviyeye getirilmesi sistemin uygulanabilirliğini ciddi oranda artırmaktadır.

BÖLÜM 2

KULLANILAN TEKNOLOJİLER VE DONANIM MİMARİSİ

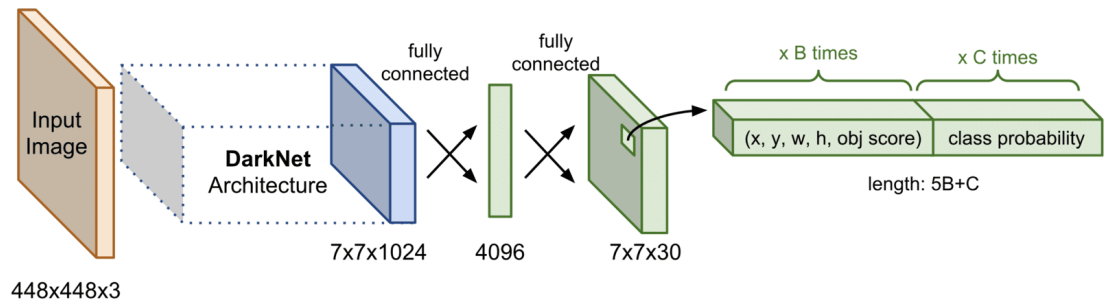
Bu projede ele alınan trafik yoğunluğu problemi, tek bir disiplinle çözülemeyecek kadar çok katmanlıdır. Bu nedenle çözüm; yapay zeka, web teknolojileri ve gömülü sistemlerin bir araya getirildiği hibrit bir mimari üzerine inşa edilmiştir. Sistemin kararlı, düşük gecikmeli ve gerçek zamanlı çalışabilmesi adına hem yazılım hem de donanım bileşenleri titizlikle belirlenmiştir.

2.1. Yazılım Teknolojileri ve Geliştirme Ortamı

Projenin yazılım iskeleti; sahadan gelen görüntülerin işlenmesi, karar algoritmalarının koşturulması ve elde edilen sonuçların sistem yöneticisine sunulması olmak üzere üç ana ayaktan oluşmaktadır.

2.1.1. Görüntü İşleme ve Yapay Zeka (YOLO ve OpenCV)

Akıllı kavşak yönetiminde en kritik faktör, sahadan akan verinin işlenme hızıdır. Kameralardan gelen görüntülerin milisaniyeler içinde anlamlandırılması gerektiği için hız ve doğruluk oranını en iyi dengeleyen YOLO (You Only Look Once) algoritması tercih edilmiştir. Şekil 2.1'de görüldüğü üzere, YOLO algoritması girdi görüntüsünü belirli bir ızgara yapısına bölerek tek bir ileri yönlü evrimsel sinir ağı üzerinden geçirir. Bu sayede, nesnelerin konumlarını belirten sınır kutuları ve sınıf olasılıkları aynı anda hesaplanarak sistemin gerçek zamanlı çalışabilmesi sağlanır.

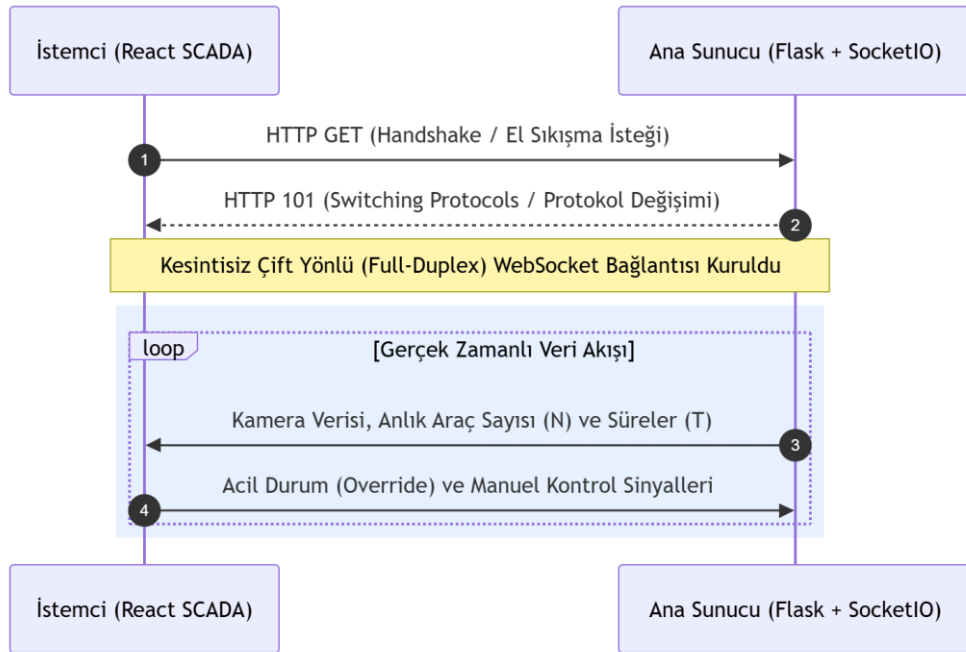


Şekil 2. 1: YOLO Algoritmasının Tek Aşamalı Nesne Tespit Mimarisi [1]

Sistemin kamera akışlarını yakalaması, çözünürlükleri yapay zekanın işleyebileceği optimum boyutlara (örneğin 640x360 piksel) getirmesi ve ağdan elde edilen bu sınır kutularının ekrana çizilmesi gibi ön işleme adımlarında ise OpenCV kütüphanesi kullanılmıştır [2]. Bu yapı sayesinde gereksiz arka plan detayları ve sahadaki ufak gürültüler (tabelalar, plakalar vb.) elenerek sadece araç sınıfına odaklanılması sağlanmıştır.

2.1.2. Arka Plan Mimarisi ve Haberleşme (Python, Flask ve SocketIO)

Sistemin merkezinde yer alan karar verici algoritma için, özellikle makine öğrenmesi kütüphanelerinin zenginliği ve geliştirme sürecine sağladığı esneklik göz önüne alınarak Python programlama dili tercih edilmiştir [3]. Ancak uygulamanın ilk aşamalarında, Python'da çalışan yapay zeka modelinin ürettiği anlık verilerin SCADA arayüzüne yansıtılmasında ciddi bir darboğazla karşılaşmıştır. Klasik REST API yapısı kullanılarak yapılan standart HTTP istekleri (sürekli veri çekme / polling mantığı), saniyelerle yarışılan bir trafik sisteminde arayüzün donmasına ve sunucuda gereksiz bir yüke sebep olmuştur.



Şekil 2.2: İstemci ve Sunucu Arasındaki Çift Yönlü Gerçek Zamanlı Veri Akışı.

Şekil 2.2'de detaylandırılan akış şemasında görüldüğü üzere, yaşanan bu iletişim gecikmesini ortadan kaldırmak amacıyla arayüzün sürekli sunucuya istek göndermesi yerine bağlantıyı daima açık tutan WebSocket protokolüne geçiş yapılmıştır. Arka planda Flask web çatısı ve SocketIO mimarisi entegre edilerek; kameralardan elde edilen aktif araç sayıları, acil durum sinyalleri ve dinamik geri sayım sürelerinin sunucu ile istemci arasında kesintisiz, çift yönlü bir şekilde akması sağlanmıştır [4]. Bu mimari sayesinde yapay zeka modelinin ürettiği kararlar, operatör ekranına neredeyse sıfır gecikmeyle yansıtılabilmektedir.

2.1.3. Kullanıcı Arayüzü (React Tabanlı SCADA)

Arka planda çalışan karmaşık yapay zeka ve sistem loglarının, sistem yöneticisine anlaşılır bir biçimde sunulması gerekmektedir. Bu amaçla, sadece güncellenen verileri (örneğin trafik sayacını) ekrana yeniden çizerek tarayıcı performansını

koruyan React kütüphanesi tercih edilmiştir [5]. Endüstriyel bir panel standartlarında tasarlanan bu arayüz; araç yoğunluğunu, panelleri ve acil durumlardaki manuel kontrol butonlarını barındırmaktadır.

2.2. Donanım Mimarisi ve Prototip Katmanı

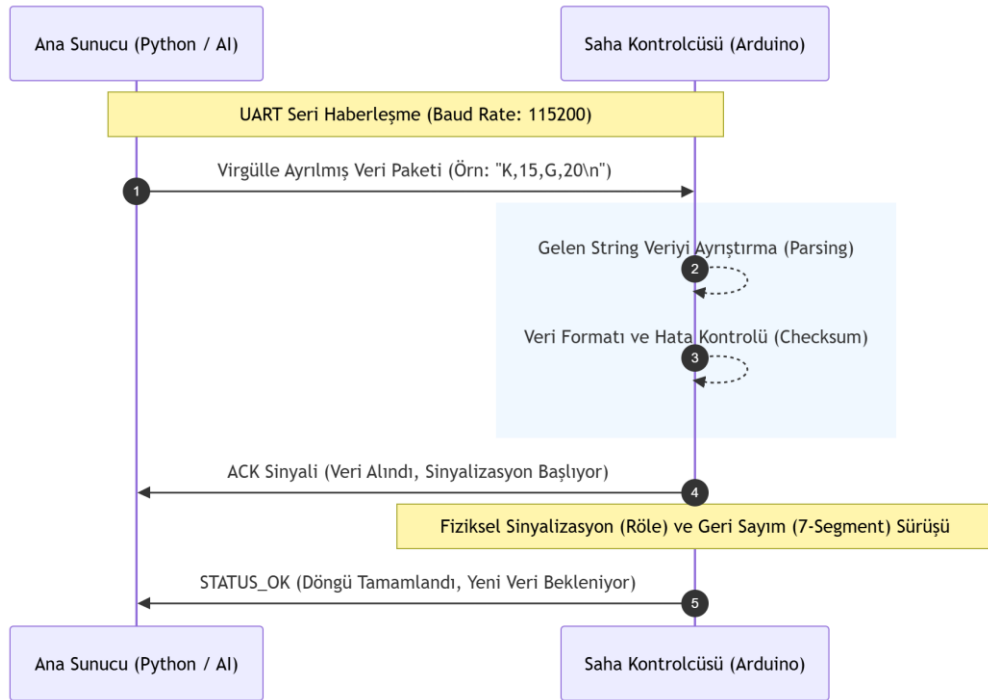
Projenin bilgisayar ortamında çalışan bir simülasyon olarak kalmaması ve sahadaki fiziksel elemanları otonom olarak yönetebilmesi amacıyla ölçeklenebilir bir donanım prototipi tasarlanmıştır.

2.2.1. Merkezi İşlem Birimi (Ana Sunucu)

Üç farklı kavşak kamerasından eşzamanlı olarak alınan yayınların YOLO modelinden geçirilmesi yüksek bir donanım gücü gerektirmektedir. Bu ağır işlem yükünü karşılamak adına, sistemin merkezine güçlü bir bilgisayar koyulmuştur. İşlemciden ziyade GPU kullanılarak video analizi saniyede yeterli kare (FPS) hızında çalıştırılabilmiştir.

2.2.2. Saha Kontrol Birimi (Mikrodenetleyici Entegrasyonu)

Yapay zekanın aldığı kararların (hangi yöne ne kadar yeşil ışık yanacağı) gerçek dünyaya aktarılması görevi, saha kontrolcüsü mantığıyla çalışan bir Arduino mikrodenetleyicisine verilmiştir..



Şekil 2. 3: Python Sunucu ve Arduino Arasındaki Veri İletişim Protokolü.

Ana karar sunucusu ile mikrodenetleyici arasındaki bağlantı UART seri haberleşme protokolü ile sağlanmıştır. Sunucu, hesapladığı dinamik ışık sürelerini virgülle ayrılmış veriler (Örn: "10,15,20") şeklinde iletirken; mikrodenetleyici bu verileri işleyip trafik ışıklarının devamlılığını sağlamaktadır.

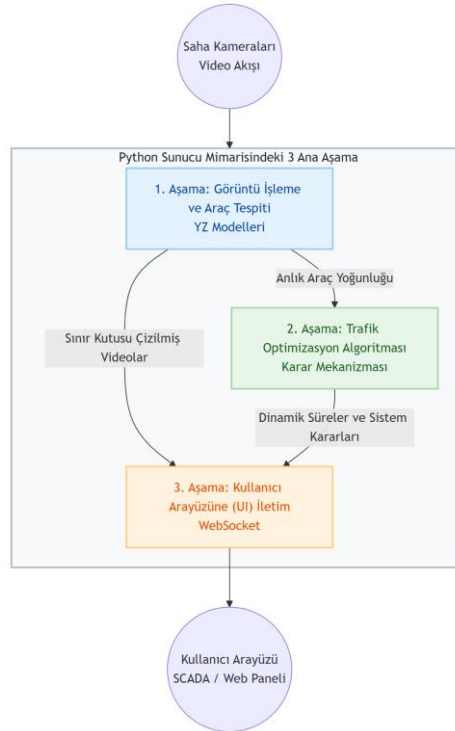
2.2.3. Endüstriyel Tip Sinyalizasyon Bileşenleri ve Sayaçlar

Trafik akışının sürücülere bildirilmesi için, prototip üzerinde standartlara yakın hazır Kırmızı-Sarı-Yeşil ışıklar kullanılmıştır. Mikrodenetleyicinin çıktılarıyla kontrol edilen bu ışıklar, kararları sürücülere hemen yansıtır.

Bunun yanı sıra sürücü güvenliğini artırmak ve kalan geçiş/bekleme sürelerini göstermek amacıyla modüle TM1637 entegreli 7-Segment ekranlar eklenmiştir. [7] Bu sayaçlar donanımsal olarak mikrodenetleyiciye bağlı çalışarak; yeşil ışık geçişini, sarı ışık beklemesini ve yönler arası güvenlik boşluklarını milisaniye hassasiyetinde geriye saymaktadır. Acil durum müdahalelerinde bu sayaçlar sunucudan gelen kesme emriyle anında sıfırlanabilmektedir.

2.3. Backend Teknolojileri

Projenin sunucu tarafı Python programlama dili ile geliştirilmiştir. Python, özellikle yapay zeka ve veri işleme kütüphaneleri açısından daha iyi olması nedeniyle tercih edilmiştir. Backend, temel olarak üç ana görevi yerine getirir: videolardan araç tespiti yapmak, bu videolara dayanarak bir trafik optimizasyon algoritması çalıştırmak ve videoları kullanıcı arayüzüne iletmek. Şekil 2.4'de bu üç aşamanın işlemleri gösterilmektedir.



Şekil 2. 4: Backend'in Üç Aşamalı Diyagramı

2.3.1. Flask ve Flask-SocketIO ile Gerçek Zamanlı Haberleşme

Sunucu altyapısı, hafif ve esnek bir Python web çatısı olan Flask üzerine kurulmuştur. Flask, React uygulamasının statik dosyalarının sunulması ve video yükleme gibi HTTP isteklerinin yönetilmesi için kullanılmıştır. Aşağıdaki kod parçasında Flask uygulamasının başlatılması ve temel yapısı görülmektedir:

```
# Flask uygulaması ve SocketIO entegrasyonu
from flask import Flask, send_from_directory
from flask_socketio import SocketIO, emit
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
STATIC_DIR = os.path.join(BASE_DIR, 'akilli-kavsak-web', 'dist')

app = Flask(__name__, static_folder=STATIC_DIR, static_url_path='')
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')
```

Şekil 2. 5: Flask Uygulamasının Başlatılması Ve Temel Yapılandırması

Ancak projenin temel ihtiyacı olan gerçek zamanlı, sürekli video akışı, Flask-SocketIO eklentisi ile sağlanmıştır. Klasik REST API yapısı kullanılarak yapılan standart HTTP istekleri (polling mantığı), saniyelerle yarışılan bir trafik sisteminde arayüzün donmasına ve sunucuda gereksiz bir yüke sebep olmuştur. Bu sorunu aşmak için bağlantıyı daima açık tutan WebSocket protokolüne geçiş yapılmıştır.

SocketIO sunucusu, bağlantı kuran her istemciye özel olaylar gönderebilmekte ve aynı zamanda istemciden gelen otomatik müdahale komutlarını çalıştırmaktadır:

```
@socketio.on('connect')
def handle_connect():
    print('✅ Client connected')
    emit('connected', {'status': 'ok'})

@socketio.on('manual_override')
def handle_manual_override(data):
    global en_yogun_index, gecis_durumu, zamanlayici_baslangic
    camera_id = data.get('camera_id', 0)

    if 0 <= camera_id < 4:
        en_yogun_index = camera_id
        gecis_durumu = False
        zamanlayici_baslangic = time.time()
```

Şekil 2. 6: İstemciden Gelen Otomatik Komutlar

2.3.2. YOLOv8 ve PyTorch ile Gerçek Zamanlı Araç Tespiti

Projenin yapay zekasını, gerçek zamanlı nesne tespiti için en güncel ve hızlı modellerden biri olan YOLOv8 algoritması oluşturmaktadır. Bu algoritma, Ultralytics firması tarafından geliştirilen ve kullanımı son derece kolay bir Python arayüzü sunan kütüphane ile geliştirilmiştir. Modelin çalıştırılması ve optimize edilmesi için derin öğrenme kütüphanesi olan PyTorch kullanılmıştır.

Sistem başlatılırken, kendi trafik veri setimizle eğitilmiş model dosyası yüklenmektedir. Eğer model bulunamazsa, sistem otomatik olarak YOLOv8'in ön eğitilmiş modeline (yolov8n.pt) geri dönüş yapmaktadır:

```
def init_model():
    global model
    TRAINED_MODEL_PATH = os.path.join(BASE_DIR, "runs", "detect",
                                       "trafik_modeli_v3", "weights", "best.pt")

    if not os.path.exists(TRAINED_MODEL_PATH):
        import glob
        pt_files = glob.glob(os.path.join(BASE_DIR, "runs", "**", "*.pt"), recursive=True)
        if pt_files:
            TRAINED_MODEL_PATH = pt_files[0]

    if os.path.exists(TRAINED_MODEL_PATH):
        model = YOLO(TRAINED_MODEL_PATH)
        return True
    else:
        model = YOLO("yolov8n.pt")
        return True
```

Şekil 2. 7: Model Dosyasının Yüklenmesi

Model yüklendikten sonra, üç farklı kavşak kamerasından gelen görüntüler işlenmektedir. Her bir kamera için ayrı bir VideoCapture nesnesi oluşturulmakta ve video akışı sürekli olarak okunmaktadır:

```
def init_videos():
    global caps, takipler
    caps = []

    for path in video_paths:
        if os.path.exists(path):
            cap = cv2.VideoCapture(path)
            caps.append(cap)
        else:
            caps.append(None)

    takipler = [MerkezTakip() for _ in range(4)]
    return True
```

Şekil 2. 8: Videoların Akışını Oluşturulması

process_frame fonksiyonu içerisinde, her bir video YOLO modelinden geçirilmekte ve tespit edilen araçlar filtrelenmektedir.

Aşağıdaki kod parçasında görüldüğü üzere, tespit edilen her nesne için güven skoru (conf) ve sınıf bilgisi (cls) işlenmekte; boyut filtrelemesi yapılarak piksellerden elde edilen küçük nesneler (plakalar, uzak araçlar vb.) aktif araç sayımına dahil edilmemektedir:

```
def process_frame(frame, camera_index):
    frame = cv2.resize(frame, (640, 360))
    results = model(frame, verbose=False, conf=0.15, iou=0.45, imgsz=640)
    r = results[0]
    aktif_arac_sayisi = 0
    annotated = frame.copy()

    if r.bboxes is not None:
        for box in r.bboxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            w = x2 - x1 # Tespit edilen nesnenin genişliği
            h = y2 - y1 # Tespit edilen nesnenin yüksekliği

            # Plaka ve gürültü filtrelemesi: 25x20 pikselden büyük nesneler sayılır
            if w > 25 and h > 20:
                aktif_arac_sayisi += 1
                cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 0), 2)

    return {
        'frame': annotated,
        'vehicles': aktif_arac_sayisi,
        'light_status': light_status,
        'light_timer': light_timer
```

Şekil 2. 9: Nesnelerin İşlenmesi ve Boyut Filtrelemesi

2.3.3. OpenCV ile Görüntü İşleme ve Base64 Dönüşümü

Görüntü işlemenin tamamı OpenCV kütüphanesi ile yönetilmektedir. Bu kütüphane; video dosyalarını okumak, kareleri boyutlandırmak, YOLO modelinden gelen koordinat bilgileriyle tespit edilen araçların etrafına sınırlayıcı kutular çizmek ve işlenmiş kareyi JPEG formatına kodlayarak base64 string'ine dönüştürmek için kullanılmıştır.

Video akışının başa sarılması ve herhangi bir hatada hata tespitinin yapılması da OpenCV üzerinden gerçekleştirilmektedir:

```
def video_akisi():
    while system_status['is_running']:
        frames = []
        for i, cap in enumerate(caps):
            if cap is not None and cap.isOpened():
                ret, frame = cap.read()
                if not ret or frame is None:
                    # Video sonuna geldiğinde başa sar
                    cap.set(cv2.CAP_PROP_POS_FRAMES, 0)
                    ret, frame = cap.read()
                if not ret or frame is None:
                    # Hata durumunda siyah bir placeholder göster
                    dummy = np.zeros((360, 640, 3), dtype=np.uint8)
                    cv2.putText(dummy, f"KAMERA {i+1} HATA", (150, 180),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
                    frames.append(dummy)
                    continue
                frames.append(frame)
```

Şekil 2. 10: Video Akışı Döngüsü Ve Hata Yönetimi

İşlenen her kare, ağ üzerinden aktarılabilmesi için önce JPEG formatına sıkıştırılmakta, ardından base64 kodlaması ile metin tabanlı bir formata dönüştürülmektedir. Bu dönüşüm, görüntünün doğrudan WebSocket üzerinden iletilmesini sağlar.

```
def frame_to_base64(frame):
    _, buffer = cv2.imencode('.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 70])
    return base64.b64encode(buffer).decode('utf-8')
```

Şekil 2. 11: Karelerin Jpeg Yapılması Ve Metin Tabanlı Formata Çevirilmesi

2.3.4. Adaptif Trafik Kontrol Algoritması (Karar Mekanizması)

YOLO'dan gelen anlık araç sayıları, karar_mekanizmasini_guncelle isimli özel bir fonksiyonda kullanılmaktadır. Bu algoritma, her bir kavşak kolundaki araç yoğunluğunu ve bekleme sürelerini sürekli takip eder.

Algoritmanın çalışma prensibi şu şekildedir: Bir kavşakta belirlenen BEKLEME_LIMITI (600 frame) aşıldığında, o kavşağa geçiş önceliği tanınır. Normal koşullarda ise en yoğun kavşak ile diğer kavşaklar arasındaki araç sayısı farkı KARAR_ESIGI değerini aştığında bir geçiş kararı alınır. Geçiş kararı alındığında, ani bir değişim yerine SARI_SURESI (3 saniye) kadar bir güvenli geçiş süreci devreye girer:

```
def karar_mekanizmasini_guncelle(arac_sayilari):
    global en_yogun_index, gecis_durumu, hedef_yogun_index
    global karar_kilidi_sayaci, zamanlayici_baslangic, bekleme_sureleri

    su_an = time.time()

    if gecis_durumu:
        gecen_sari_suresi = su_an - zamanlayici_baslangic
        if gecen_sari_suresi >= SARI_SURESI:
            en_yogun_index = hedef_yogun_index
            gecis_durumu = False
            zamanlayici_baslangic = time.time()
            karar_kilidi_sayaci = MIN_YESIL_SURESI
            bekleme_sureleri[en_yogun_index] = 0
        else:
            if karar_kilidi_sayaci > 0:
                karar_kilidi_sayaci -= 1
            else:
                istenen_index = en_yogun_index
                ac_kalanlar = [i for i in range(4) if bekleme_sureleri[i] > BEKLEME_LIMITI]

                if ac_kalanlar:
                    istenen_index = ac_kalanlar[0]
                else:
                    potansiyel_yeni_index = np.argmax(arac_sayilari)
                    if arac_sayilari[potansiyel_yeni_index] > (arac_sayilari[en_yogun_index] + KARAR_ESIGI):
                        istenen_index = potansiyel_yeni_index

                if istenen_index != en_yogun_index:
                    gecis_durumu = True
                    hedef_yogun_index = istenen_index
                    zamanlayici_baslangic = time.time()
                    print(f"🟡 Işık değişiyor: {kamera_isimleri[en_yogun_index]} -> {kamera_isimleri[hedef_yogun_index]}")
```

Şekil 2. 12: Limitler Aşıldığında, Sarı Işıklı Güvenli Geçiş Başlatma Fonksiyonu

Sistem, MIN_YESIL_SURESI (100 frame) kadar bir kilitlenme (karar_kilidi_sayaci) uygulayarak sık ve gereksiz yön değişimlerinin önüne geçmektedir. Bu mekanizma, sabit süreli sistemlerin aksine, trafiğin dinamik yapısına adapte olan zeki bir fikir sunar.

2.3.5. MerkezTakip Sınıfı ile Hareket Analizi

MerkezTakip sınıfı, araçların hareketlilik durumunu takip ederek kırmızı ışıktaki bekleyen ve tamamen duran araçları ayırt etmeye yarayan bir sayım mekanizması olarak tasarlanmıştır. Bu sınıf, her bir tespit kutusunun merkez noktasını hesaplayarak ardışık kareler arasında eşleştirme yapar:

```
class MerkezTakip:
    def __init__(self, tolerans_piksel=40, park_siniri=50):
        self.tolerans = tolerans_piksel
        self.park_siniri = park_siniri
        self.hafiza = []

    def hareketli_araclari_say(self, yeni_kutular, isik_kirmizi=False):
        if yeni_kutular is None or len(yeni_kutular) == 0:
            return 0

        hareketli_sayi = 0
        yeni_hafiza = []

        for kutu in yeni_kutular:
            try:
                x1, y1, x2, y2 = kutu.xyxy[0]
                cx, cy = int((x1 + x2) / 2), int((y1 + y2) / 2)

                eslesti_mi = False
                for eski in self.hafiza:
                    mesafe = math.hypot(cx - eski['merkez'][0], cy - eski['merkez'][1])

                    if mesafe < self.tolerans:
                        eslesti_mi = True
                        if isik_kirmizi:
                            yeni_sayac = eski['sayac']
                        else:
                            yeni_sayac = eski['sayac'] + 1

                yeni_hafiza.append({'merkez': (cx, cy), 'sayac': yeni_sayac})

                if yeni_sayac < self.park_siniri:
                    hareketli_sayi += 1
                    break
```

Şekil 2. 13: Araç Takibi Ve Sayım Mekanizması

Bir aracın merkez noktası tolerans mesafesi içinde birden fazla kare boyunca aynı konumda kalıyorsa, o araç "park etmiş" olarak sınıflandırılır ve hareketli araç sayımına dahil edilmez.

2.3.6. Video İşleme Ana Döngüsü ve Veri İletimi

Tüm bu bileşenler, video_akisi isimli ana fonksiyonda bir araya getirilmektedir. Bu fonksiyon, ayrı bir iş parçacığında (thread) çalışarak ana Flask uygulamasını bloke etmeden sürekli olarak video işleme döngüsünü yürütür.

Ana döngü içerisinde, üç kameradan gelen kareler sırayla işlenmekte, karar mekanizması güncellenmekte ve sonuçlar SocketIO üzerinden istemciye iletilmektedir:

```

def video_akisi():
    global system_status

    while system_status['is_running']:
        # 1. Aşama: Tüm kameralardan kareleri oku
        frames = []
        for i, cap in enumerate(caps):
            # ... kare okuma işlemleri ...

        # 2. Aşama: Her kareyi YOLO'dan geçir ve araç say
        camera_data = []
        arac_sayilari = []
        for i, frame in enumerate(frames):
            processed = process_frame(frame, i)
            camera_data.append(processed)
            arac_sayilari.append(processed['vehicles'])

        # 3. Aşama: Karar mekanizmasını güncelle
        karar_mekanizmasini_guncelle(arac_sayilari)

```

Şekil 2. 14: Karar Mekanizması Güncelleme

Sistem, saniyede yaklaşık 30 kez (0.033 saniyelik uyku süresi) bu döngüyü tekrarlayarak gerçek zamanlıya yakın bir performans sunmaktadır. İşlenen her kare, base64 formatına dönüştürüldükten sonra camera_update olayı ile; sistem metrikleri ise system_update olayı ile istemciye gönderilmektedir:

```

socketio.emit('system_update', {
    'active_vehicles': total_vehicles,
    'algorithm_status': 'ACTIVE',
    'avg_wait_time': round(sum(bekleme_sureleri) / 4, 1),
    'throughput': total_vehicles * 12
})

```

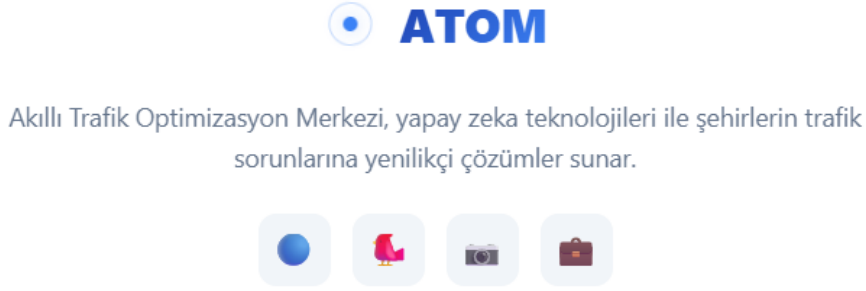
Şekil 2. 15: İşlenen Karelerin İstemciye Gönderilmesi

2.4. Frontend Teknolojileri

Projenin kullanıcı arayüzü (Front-End) geliştirilirken, sistemin sadece bir laboratuvar simülasyonu olarak kalmaması ve gerçek dünyada belediyeler veya trafik kontrol merkezleri tarafından doğrudan kullanılacak kurumsal bir ürün (SaaS) formatında kurgulanması hedeflenmiştir. Bu doğrultuda sisteme ATOM (Akıllı Trafik Optimizasyon Merkezi) adı verilmiş ve modern web teknolojileri kullanılarak kullanıcı dostu bir arayüz inşa edilmiştir. Saniyede onlarca kez güncellenen kamera akışlarının ve trafik metriklerinin tarayıcıyı yormadan ekrana yansıtılabilmesi için arayüz mimarisinin temelinde React.js kütüphanesi kullanılmıştır.

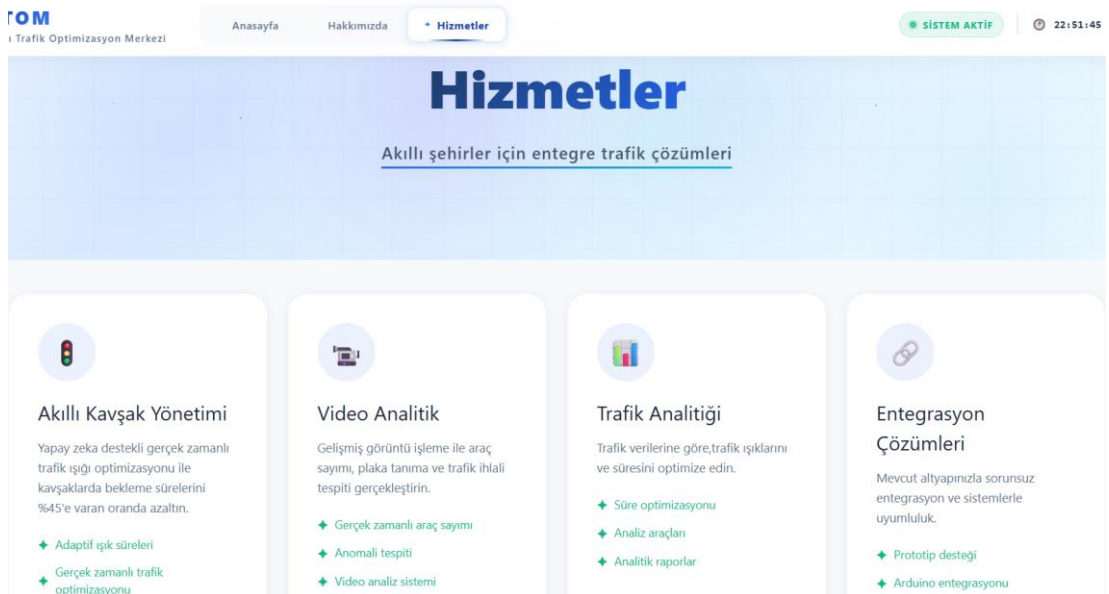
2.4.1. Kurumsal Kimlik ve Bilgilendirme Ekranları

Uygulama arayüzü, sadece teknik verilerin aktığı karmaşık bir panel olmak yerine, sistemin amacını ve kapasitesini anlatan modüler sayfalarla (Anasayfa, Hakkımızda, Hizmetler) desteklenmiştir.

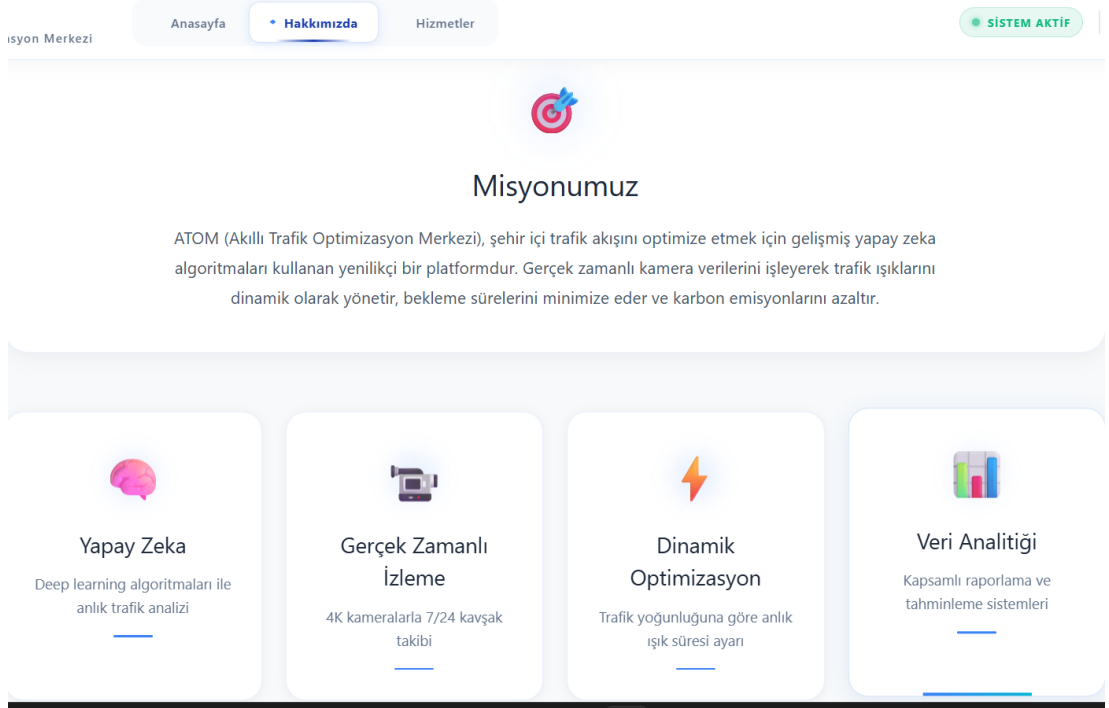


Şekil 2. 16: Kimlik Ve Bilgilendirme Ekranı

Sistemin sunduğu "Gerçek Zamanlı İzleme", "Dinamik Optimizasyon" ve "Video Analitik" gibi yetenekler, kullanıcıların (veya yöneticilerin) sistemi kolayca kavrayabilmesi için arayüzde kart yapıları halinde görselleştirilmiştir. Ayrıca "Hakkımızda" bölümünde projenin temel iddiaları olan %99.9 çalışma süresi, %45 verimlilik artışı ve 7/24 kesintisiz izleme gibi performans hedefleri dinamik grafiklerle sunulmuştur. Bu yapı, projenin tek bir amaca hizmet eden bir kod parçasından ziyade, ölçeklenebilir ve entegre edilebilir bir akıllı şehir çözümü olduğunu vurgulamaktadır.



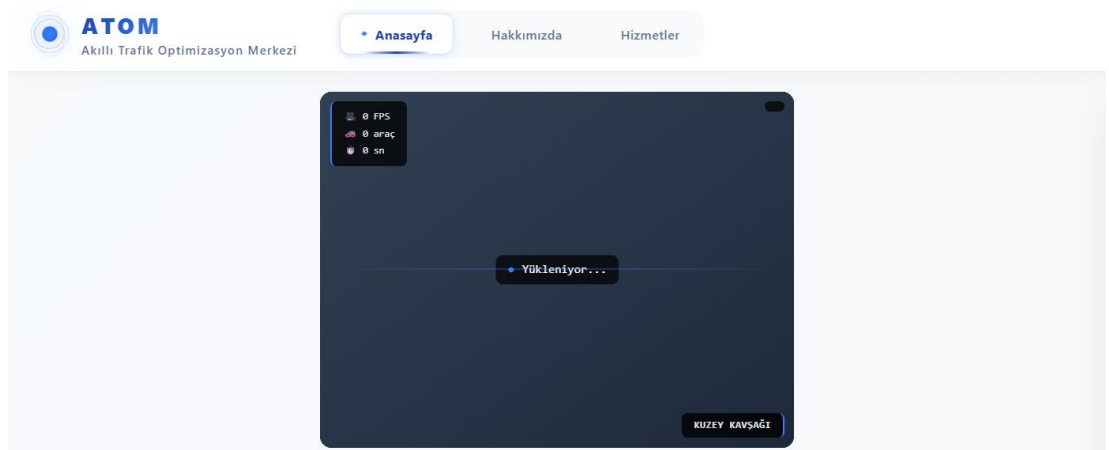
Şekil 2. 17: Hizmetler Ekranı



Şekil 2. 18: Hakkımızdanın İçeriğindeki Misyonumuz Ekranı

2.4.2. Gerçek Zamanlı Kamera Paneli ve Optimizasyon

Trafik yönetim merkezindeki operatörlerin uzun saatler boyunca ekran başında kalacağı göz önüne alınarak, operasyonel izleme sayfalarında göz yorgunluğunu minimize eden koyu temalı bir tasarım dili tercih edilmiştir.



Şekil 2. 19: Kamera İçeriği Arayüzü

Kamera paneli (Şekil 2.19), geleneksel video oynatıcılardan farklı bir mantıkla kurgulanmıştır. Arka planda kurulan Socket.IO bağlantısı sayesinde sunucudan Base64 formatında gelen ve saniyede yaklaşık 30 kez yenilenen işlenmiş görüntü

dizgileri, hiçbir video dosyası belleğe indirilmeden anlık olarak render edilmektedir. Görselde de görüldüğü üzere, videonun sol üst köşesine yerleştirilen otonom bir modül sayesinde o anki FPS değeri, tespit edilen aktif araç sayısı ve yönün bekleme süresi operatöre doğrudan video üzerinden sunulmaktadır. Kamera bağlantısı kurulana kadar veya bağlantı koptuğunda arayüzün bozulmaması için zarif bir "Yükleniyor..." Durumu tasarımı eklenmiştir.

2.4.3. Sistem Metrikleri ve Operatör Kontrol Modülü

Arayüz, sadece bir "izleme" ekranı değil, aynı zamanda interaktif bir "müdahale ve durum" paneli olarak kurgulanmıştır.



Şekil 2. 20: Akıllı Kontrol Merkezi Menüsü

Şekil 2.20'de görülen "Akıllı Kavşak Merkezi" panosu, arka planda çalışan yapay zeka algoritmasının canlı bir röntgeni niteliğindedir. Bu panelde React'ın durum

yönetimi etkin bir şekilde kullanılmış ve güncellemeler sadece değerler değiştiğinde tetiklenecek şekilde optimize edilmiştir. Panel üzerindeki temel işlevler şunlardır:

Canlı Metrikler: Aktif araç sayısı, ortalama bekleme süresi ve saatlik araç akışı gibi veriler milisaniye gecikmeyle güncellenmektedir.

Algoritma Durumu: Arka plandaki YOLO ve karar algoritmasının "ACTIVE" olarak devrede olup olmadığı buradan takip edilmektedir.

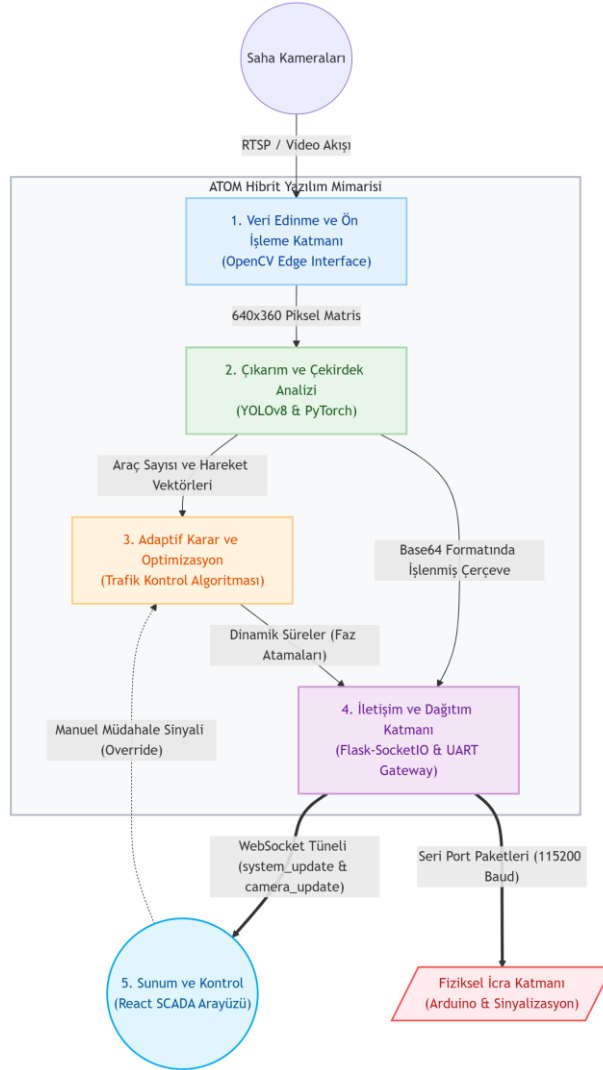
Manuel Müdahale (Sistemi Başlat): Sistemin otonom yapay zeka kararlarına bırakılmadan önce veya acil durumlarda sistem yöneticisi tarafından manuel olarak tetiklenebilmesi için panele eyleme geçirici bir buton yerleştirilmiştir. Bu buton, insan-makine etkileşimini sağlayarak sistemin kontrolsüz bir şekilde donanımı sürmesini engellemekte ve son onayı sistem yöneticisine bırakmaktadır

BÖLÜM 3

YAZILIM MİMARİSİ

ATOM (Akıllı Trafik Optimizasyon Merkezi) sisteminin mimari tasarımı; yüksek işlem yükü gerektiren yapay zeka çıkarımlarının, web sitesinin ve fiziksel saha donanımının birbirini bloklamadan, eş zamanlı çalışabilmesi esasına dayanır. Sistemde tek bir noktada yaşanacak bir gecikmenin veya çökmenin tüm trafik akışını kesintiye uğratmasını engellemek adına, bileşenlerin gevşek bağlı olarak çalıştığı, katmanlı ve bir hibrit yazılım mimarisi tercih edilmiştir.

Sistemin katmanlı yapı şeması ve bileşenler arası ilişkiler Şekil 3.1’de modellenmiştir.



Şekil 3. 1: Sistemin Mimarisinin Tasarımı

Yazılım mimarisini oluşturan temel katmanlar ve bunların mimari içindeki işlevler:

3.1. Veri Edinme ve Ön İşleme Katmanı (OpenCV Edge Interface)

Sistemin giriş kapısı olan bu katman, sahada yer alan üç bağımsız kavşak kameralarının akışlarını ya da video kaynaklarını yakalamakla görevlidir. Mimari açıdan en büyük kısıt, yüksek çözünürlüklü video verisinin sistem belleğinde yaratacağı şişmedir. Bu darboğazı engellemek için, gelen her bir kare işlemcide belirli bir piksel boyutuna indirgenerek bir sonraki katmana aktarılır.

3.2. Çıkarım ve Çekirdek Analiz Katmanı (YOLOv8 Core & PyTorch)

Yazılımı oluşturan bu katmanda, CUDA çekirdekleri üzerinde koşturulan YOLOv8 nesne tespiti modeli yer alır. Mimari tasarımda modelin her kareyi yakalamak için ana uygulama döngüsünü kilitlemesini önlemek amacıyla, video işleme süreçleri bağımsız bir iş parçası olarak kullanılmıştır. process_frame fonksiyonu, gelen verilerini işleyerek araç koordinatlarını çıkarır ve eş zamanlı olarak MerkezTakip sınıfı üzerinden nesnelerin hareketlerini analiz eder. Bu katman, ham görsel veriyi sayısal değerlere (araç sayısı, hareketlilik durumu) dönüştürerek sistem yükünü azaltmaya çalışır.

3.3. Adaptif Karar ve Optimizasyon Katmanı (Control Logic)

Çekirdekten gelen sayısal veriler, bu katmanda yer alan karar_mekanizmasını_guncelle algoritmasında kullanılır. Mimari açıdan bu katman, sistemin "karar verici" mekanizmasıdır. Gelen anlık araç sayıları ve hafızada tutulan bekleme_sureleri dizisi işlenerek, BEKLEME_LIMITI ve KARAR_ESIGI kısıtları altında bir durum makinesi mantığı çalıştırılır. Algoritma, kavşağın o anki ışık durumunu (Yeşil, Sarı, Kırmızı) belirler ve bir sonraki katmana emirleri üretir.

3.4. İletişim ve Dağıtım Katmanı (Flask-SocketIO & UART Gateway)

Üretilen kararların ve kamera görüntülerinin hem kullanıcı arayüzüne hem de donanıma dağıtılması bu katmanın sorumluluğundadır. Mimari, verinin niteliğine göre iki farklı iletişim kapısı kullanır:

WebSocket Protokolü (SocketIO): İşlenen görüntülerin base64 string formatına dönüştürülmüş hali ve anlık sistem metrikleri, HTTP protokolünün getirdiği bağlantı maliyetlerinden kaçınmak amacıyla camera_update ve system_update olayları üzerinden React istemcisine anlık olarak gönderilir.

Seri Haberleşme (UART): Algoritmanın ürettiği kavşak süreleri ve faz bilgileri, belirli bir hızda çalışan bir seri port tüneli üzerinden virgüllerle ayrılmış paketler halinde fiziksel saha kontrolcüsüne (Arduino) iletilir.

3.5. Sunum ve Kontrol Katmanı (React SCADA)

Kullanıcı arayüzü, Virtual DOM mimarisi sayesinde sunucudan saniyede 30 kez gelen verileri sayfayı yenilemeden karşılar. Mimari tasarımda her bir bileşen (kamera akış penceresi, metrik kartları, kontrol butonları) birbirinden izole edilmiş birer React

parçası olarak kurgulanmıştır. Bu sayede sadece verisi değişen alanlar ekrana yeniden çizilir ve operatörün kullandığı tarayıcının kilitlenmesi önlenir. Ayrıca operatörün gönderdiği `manual_override` komutları da aynı hat üzerinden geriye doğru akarak karar katmanını anında değiştirebilecek esnekliktedir.

BÖLÜM 4

ANOMALİ TESPİT SÜRECİ

Geliştirilen akıllı kavşak sisteminin sahadaki başarısı, yalnızca normal trafik akışını sayıp ışık sürelerini ayarlamasıyla değil; olağandışı durumlara ne kadar hızlı ve doğru tepki verebildiğiyle ölçülmektedir. Sistem sadece bir sayaç olarak kalmamalı, aynı zamanda yolu izleyen bir güvenlik görevlisi gibi davranmalıydı. Bu amaçla, ana görüntü işleme döngüsünün içerisine otonom bir anomali tespit algoritması entegre edilmiştir.

4.1. Hareketsiz Araç ve Kaza Algılama Mantığı

Kavşak içindeki olası bir kazayı veya arızalanıp yolda kalan bir aracı tespit etmenin en güvenilir yolu, araçların hareket vektörlerini incelemektir. Ancak bu aşamada, sistemi geliştirirken mantıksal bir zorlukla karşılaşıldı: Sistem, kırmızı ışıkta yasal olarak bekleyen araçları da "hareketsiz" olarak algılayıp sürekli yanlış kaza alarmı üretiyordu.

Bu sorunu çözmek için araç takip algoritması ile trafik ışıklarının anlık durumu birbirine bağlandı. Sistemin bir aracı anomali olarak işaretlemesi için şu kural seti oluşturuldu: İlgili kavşak kolunda yeşil ışık yanıyor olmasına rağmen, bir aracın merkez koordinatları önceden belirlenen piksel tolerans mesafesi dışına çıkmıyorsa ve bu durum belli bir süre devam ediyorsa, sistem o aracı yolda kalmış veya kaza yapmış olarak sınıflandırır. Bu sayede trafik ışığı kırmızı iken oluşan doğal kuyruklar hata olarak algılanmaktan çıkarıldı ve algoritmanın doğruluğu sağlandı.

4.2. Acil Durum Araçlarının Tanınması ve Önceliklendirme

Trafik yönetiminin en kritik aşamalarından biri olan geçiş üstünlüğüne sahip araçların yönetimi, projeye özel bir modül olarak eklenmiştir. Yapay zeka modeli eğitilirken ambulans ve itfaiye gibi araçlar standart sivil araçlardan ayrı bir sınıf olarak etiketlenmiştir.

Kameralardan herhangi biri bir acil durum aracı tespit ettiğinde, arka planda çalışan standart yoğunluk hesaplama algoritması derhal askıya alınır. Sistem, acil durum aracının bulunduğu yöne doğrudan yeşil ışık yakmak üzere bir zorunlu geçiş sinyali üretir. Diğer tüm yönler, kavşağın hızla boşalması için güvenli bir şekilde kırmızı ışığa çekilir. Araç kamerasının görüş açısından tamamen çıkana kadar sistem bu acil durum modunda kilitli kalır ve ardından normal işleyişine kaldığı yerden, bekleme sürelerini sıfırlamadan devam eder.

4.3. Yanlıř Alarmların Filtrelenmesi ve Kararlılık

Saha ortamı; gece oluřan far parlamaları, řiddetli yaęmur, kameranın lensine dūřen yansımalar veya yola giren sokak hayvanları gibi sisteme anomali olarak yansıyabilecek sayısız gürültü barındırır. Projenin ilk testlerinde, kameranın önünden geen büyük bir kuřun veya saniyelik bir görüntü bozulmasının bile arayüzde kırmızı alarm oluřturduęu gözlemlenmiřtir.

Bu aşırı hassasiyeti ve gereksiz panięi önlemek adına karar mekanizmasına bir zaman filtresi eklenmiřtir. Yapay zeka bir karede anomali tespit etse bile bunu hemen onaylamaz; aynı olaęandışı durumun art arda belirli sayıda kare boyunca kesintisiz devam etmesini bekler. Bu bekleme süresi, saniyelik hataları filtreleyecek kadar uzun, gerek bir kazaya anında müdahale edebilecek kadar kısa tutulmuřtur. Kesinleřen anomaliler, derhal kontrol merkezindeki arayüze kırmızı bir uyarı olarak iletilir ve son takdir tamamen ekran bařı

BÖLÜM 5

TESTLER VE SONUÇLAR

Bu bölümde geliştirilen görüntü işleme destekli otonom kavşak ve trafik yönetim sisteminin farklı senaryolar altında gerçekleştirilen test sonuçları sunulmuştur. Sistemin araç tespit performansı, trafik ışığı kontrol mekanizması, Arduino ile haberleşme yapısı ve manuel çalışma modu ayrı ayrı değerlendirilmiştir.

5.1 Araç Yoğunluğu Tespit Testleri

Sistemin temel çalışma mantığı, kavşaklardan alınan görüntüler üzerinde araç yoğunluğunun doğru şekilde tespit edilmesine dayanmaktadır. Bu amaçla farklı yoğunluk seviyelerine sahip video kayıtları kullanılarak çeşitli testler gerçekleştirilmiştir. Testlerde YOLO tabanlı nesne tespit modeli kullanılmış ve görüntüler üzerinde bulunan araçlar anlık olarak sayılmıştır.

Yapılan denemelerde sistemin araçları başarılı şekilde tespit ettiği ve yoğunluk değişimlerine hızlı şekilde tepki verdiği gözlemlenmiştir. Özellikle yoğun trafik senaryolarında araç sayılarının doğru şekilde belirlenmesi sayesinde sistemin karar mekanizmasının sağlıklı çalıştığı görülmüştür. Bununla birlikte düşük ışık koşulları ve görüntü kalitesinin düştüğü bazı durumlarda algılama performansında kısmi düşüşler yaşanmıştır.

5.2 Trafik Işığı Sürelerinin Dinamik Olarak Ayarlanması

Araç yoğunluğu bilgileri elde edildikten sonra trafik ışık süreleri dinamik olarak hesaplanmıştır. Sistem, yoğunluğun fazla olduğu yönlere daha uzun yeşil ışık süresi verirken yoğunluğun az olduğu yönlerde bekleme sürelerini azaltmıştır.

Gerçekleştirilen testlerde yoğunluğun yüksek olduğu kavşaklarda araç kuyruklarının daha hızlı eritildiği gözlemlenmiştir. Sabit zamanlı sistemlerle karşılaştırıldığında bekleme sürelerinde azalma sağlandığı ve trafik akışının daha dengeli hale geldiği görülmüştür. Bu sonuçlar sistemin temel hedeflerinden biri olan adaptif trafik yönetiminin başarılı şekilde gerçekleştirildiğini göstermektedir.

5.3 Arduino ve Seri Haberleşme Testleri

Python tarafında çalışan görüntü işleme sistemi ile Arduino UNO arasında seri haberleşme kullanılmıştır. Araç yoğunluğuna göre hesaplanan trafik süreleri seri port üzerinden Arduino kartına aktarılmıştır.

Yapılan testlerde veri iletiminde herhangi bir senkronizasyon problemi yaşanmamış ve Arduino tarafından alınan veriler başarıyla işlenmiştir. Trafik lambaları ve geri sayım ekranları gönderilen süre bilgilerine uygun şekilde çalışmıştır. Ayrıca sistem kapatıldığında LED modüllerinin ve göstergelerin güvenli şekilde başlangıç durumuna dönmesi sağlanmıştır.

5.4 Sistemin Manuel Modda Çalıştırılması

Sistem içerisinde olası görüntü işleme hataları veya beklenmedik durumlar için manuel çalışma modu geliştirilmiştir. Bu mod sayesinde görüntü işleme sistemi devre dışı kalsa bile trafik ışıkları önceden belirlenen sürelerle çalışmaya devam etmektedir.

Gerçekleştirilen testlerde manuel moda geçiş işlemi başarıyla uygulanmış ve sistemin kesintisiz şekilde çalışmaya devam ettiği görülmüştür. Bu özellik sistemin güvenilirliğini artıran önemli bileşenlerden biri olmuştur.

5.5 Genel Değerlendirme

Gerçekleştirilen testler sonucunda geliştirilen sistemin temel işlevlerini başarılı şekilde yerine getirdiği görülmüştür. Araç yoğunluğu doğru şekilde tespit edilmiş, trafik ışıkları yoğunluğa göre dinamik olarak kontrol edilmiş ve donanım ile yazılım bileşenleri arasında kararlı bir iletişim sağlanmıştır.

Elde edilen sonuçlar, görüntü işleme destekli trafik yönetim sistemlerinin geleneksel sabit zamanlı sistemlere göre daha verimli çalışabileceğini göstermektedir. Prototip ölçeğinde geliştirilen bu sistem, daha gelişmiş donanım ve gerçek trafik verileri ile desteklenmesi durumunda gerçek hayat uygulamalarına uyarlanabilecek bir altyapı sunmaktadır.

BÖLÜM 6

SONUÇ VE GELECEK ÖNERİLER

6.1 Projenin Başlıca Kazanımları

Bu tez çalışmasında görüntü işleme destekli otonom bir kavşak ve trafik yönetim sistemi geliştirilmiştir. Geliştirilen sistemde kavşaklardan alınan görüntüler gerçek zamanlı olarak işlenmiş, araç yoğunlukları belirlenmiş ve elde edilen verilere göre trafik ışık süreleri dinamik olarak ayarlanmıştır. Böylece sabit zamanlı trafik ışıklarının neden olduğu gereksiz beklemlerin azaltılması amaçlanmıştır.

Proje kapsamında YOLO tabanlı nesne tespit algoritması kullanılarak araçlar anlık olarak tespit edilmiş ve yoğunluk bilgileri elde edilmiştir. Python ortamında çalışan karar mekanizması, yoğunluk verilerini değerlendirerek trafik ışıklarının çalışma sürelerini belirlemiştir. Hesaplanan süreler seri haberleşme aracılığıyla Arduino tabanlı kontrol sistemine aktarılmış ve trafik lambaları ile geri sayım ekranları bu verilere göre yönetilmiştir.

Geliştirilen web tabanlı izleme arayüzü sayesinde sistemin durumu gerçek zamanlı olarak takip edilebilmiş ve gerekli durumlarda kullanıcı müdahalesine olanak sağlanmıştır. Ayrıca sistemde oluşturulan manuel çalışma modu sayesinde olası yazılım veya görüntü işleme hatalarında trafik ışıklarının çalışmaya devam etmesi sağlanmıştır.

Yapılan testler sonucunda sistemin araç yoğunluğuna göre karar verebildiği, trafik ışıklarını dinamik olarak yönetebildiği ve donanım ile yazılım bileşenleri arasında kararlı bir iletişim kurabildiği görülmüştür. Elde edilen sonuçlar, görüntü işleme tabanlı trafik yönetim sistemlerinin trafik akışının iyileştirilmesinde etkili bir çözüm olabileceğini göstermektedir.

6.2 Gelecek Çalışmalar İçin Öneriler

6.2.1 Acil Durum Araçlarına Öncelik Verilmesi

Gelecekte sistem içerisine ambulans, itfaiye ve polis araçlarını tanıyabilecek yapay zeka modelleri entegre edilebilir. Böylece acil durum araçlarının kavşaklardan daha hızlı geçmesi sağlanabilir ve müdahale süreleri azaltılabilir.

6.2.2 Gelişmiş Araç Sınıflandırma Sistemleri

Mevcut sistem araç yoğunluğunu temel almaktadır. İlerleyen çalışmalarda araçlar otomobil, motosiklet, otobüs ve ağır vasıta gibi sınıflara ayrılarak daha hassas trafik optimizasyonu gerçekleştirilebilir.

6.2.3 Gerçek Trafik Kameraları ile Entegrasyon

Bu çalışmada kullanılan test videoları yerine gerçek trafik kameraları kullanılarak sistem gerçek şehir ortamlarında uygulanabilir hale getirilebilir. Bu sayede farklı hava koşulları ve trafik senaryoları altında sistem performansı değerlendirilebilir.

6.2.4 Merkezi Trafik Yönetim Altyapısı

Birden fazla kavşağın tek bir merkezden yönetilmesi sağlanarak şehir genelinde koordineli trafik kontrolü gerçekleştirilebilir. Böylece kavşaklar birbirleriyle veri paylaşarak daha verimli trafik akışı oluşturabilir.

6.2.5 Mobil ve Web Tabanlı Yönetim Sistemleri

Sisteme mobil uygulama desteği eklenerek trafik yoğunluklarının ve kavşak durumlarının uzaktan takip edilmesi mümkün hale getirilebilir. Ayrıca sistem yöneticilerine anlık bildirimler gönderilerek daha hızlı müdahale imkânı sağlanabilir.

6.3 Sistemin Geliştirilebilir Yönleri

6.3.1 Kamera ve Algılama Performansının Artırılması

Daha yüksek çözünürlüklü kameralar ve gelişmiş görüntü işleme algoritmaları kullanılarak araç algılama doğruluğu artırılabilir. Özellikle gece koşullarında ve olumsuz hava şartlarında daha başarılı sonuçlar elde edilebilir.

6.3.2 Büyük Ölçekli Kavşak Yapılarının Desteklenmesi

Prototip olarak geliştirilen sistem daha fazla kavşak ve trafik noktası içeren büyük ölçekli yapılara uyarlanabilir. Böylece sistemin ölçeklenebilirliği test edilmiş olur.

6.3.3 Bulut Tabanlı Veri Analizi

Toplanan trafik verileri bulut ortamında depolanarak uzun dönemli analizler yapılabilir. Elde edilen veriler sayesinde yoğunluk tahminleri oluşturulabilir ve trafik yönetimi daha proaktif hale getirilebilir.

6.3.4 Akıllı Şehir Sistemleri ile Entegrasyon

Geliştirilen sistem, gelecekte akıllı şehir uygulamalarının bir parçası olarak kullanılabilir. Trafik yönetimi, toplu taşıma sistemleri ve güvenlik altyapıları ile entegre çalışarak daha kapsamlı çözümler sunabilir.

KAYNAKÇA

- [1] J. Redmon, S. Divvala, R. Girshick ve A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, ss. 779-788.
- [2] G. Bradski, “The OpenCV Library,” *Dr. Dobb's Journal of Software Tools*, 2000.
- [3] Python Software Foundation, *Python Documentation*. Available: <https://docs.python.org/3/>
- [4] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed., O'Reilly Media, 2018.
- [5] Socket.IO, *Socket.IO Documentation*. Available: <https://socket.io/>
- [6] Meta Platforms Inc., *React Documentation*. Available: <https://react.dev/>
- [7] Arduino, *Arduino UNO Rev3 Technical Specifications*. Available: <https://www.arduino.cc/>
- [8] Titan Micro Electronics, *TM1637 LED Driver Control Circuit Datasheet*, Shenzhen, China.
- [9] A. Paszke, S. Gross, F. Massa ve diğerleri, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [10] A. Bochkovskiy, C. Y. Wang ve H. Y. M. Liao, “YOLOv4: Optimal Speed and Accuracy of Object Detection,” *arXiv preprint arXiv:2004.10934*, 2020.

EK-1 Risk Yönetim Tablosu

RİSK YÖNETİMİ TABLOSU

İP No	En Önemli Risk(ler)	B Planı
İP 1	YOLO modelinin araçları doğru tespit edememesi	Farklı model ağırlıkları kullanılarak yeniden eğitim veya parametre optimizasyonu yapılacaktır.
İP 2	Kamera görüntüsünün alınamaması veya görüntü kalitesinin düşmesi	Kayıtlı test videoları kullanılarak sistem çalıştırılacaktır.
İP 3	Python ile Arduino arasındaki seri haberleşmenin kesilmesi	Arduino üzerinde sabit zamanlı trafik ışığı modu devreye alınacaktır.
İP 4	Sistem performansının gerçek zamanlı çalışmaya yetmemesi	Görüntü çözünürlükleri düşürülerek ve işlem optimizasyonları yapılarak performans artırılacaktır.
İP 5	Donanım bileşenlerinde (Arduino, LED, TM1637 vb.) arıza oluşması	Yedek donanım bileşenleri kullanılarak sistem çalışmaya devam edecektir.

EK-2 Standartlar ve Kısıtlar Formu

STANDARTLAR VE KISITLAR FORMU

Bitirme Tezinin hazırlanmasında Standart ve Kısıtlarla ilgili olarak, aşağıdaki soruları cevaplayınız.

1. Projenizin tasarım boyutu nedir?

Bu projede görüntü işleme, yapay zeka, web teknolojileri ve gömülü sistemler bir arada kullanılmıştır. Amaç, kavşaklardaki araç yoğunluğunu analiz ederek trafik ışıklarını otomatik olarak yönetebilen bir sistem geliştirmektir. Bu nedenle proje hem yazılım hem de donanım tasarımını içeren çok yönlü bir çalışma olmuştur.

2. Projenizde bir mühendislik problemini kendiniz analiz edip, çözüm geliştirdiniz mi?

Evet. Trafik ışıklarının sabit sürelerle çalışmasının bazı durumlarda gereksiz bekleme sürelerine neden olduğu gözlemlenmiştir. Bu problem analiz edilerek araç yoğunluğunu kameralar yardımıyla tespit eden ve trafik ışık sürelerini buna göre ayarlayan bir sistem geliştirilmiştir.

3. Önceki derslerde edindiğiniz hangi bilgi ve becerileri kullandınız?

Proje geliştirilirken programlama, görüntü işleme, yapay zeka, veri yapıları, gömülü sistemler ve bilgisayar ağları derslerinde öğrenilen bilgilerden yararlanılmıştır. Ayrıca ekip çalışması ve problem çözme becerileri de proje sürecinde aktif olarak kullanılmıştır.

4. Kullandığınız veya dikkate aldığınız mühendislik standartları nelerdir?

Projede Python-Arduino haberleşmesi için UART seri haberleşme yapısı kullanılmıştır. Web tarafında ise HTTP ve WebSocket tabanlı haberleşme yöntemlerinden yararlanılmıştır. Trafik ışıklarının çalışma mantığında ise temel trafik sinyalizasyon kuralları dikkate alınmıştır.

a) Ekonomi: Proje geliştirilirken maliyetin düşük tutulmasına dikkat edilmiştir. Bu nedenle Arduino UNO, TM1637 ekran modülleri ve hazır trafik ışığı modülleri gibi uygun maliyetli bileşenler tercih edilmiştir.

b) Çevre Sorunları: Trafik yoğunluğunun azaltılması sayesinde araçların gereksiz bekleme sürelerinin düşürülmesi ve buna bağlı olarak yakıt tüketimi ile egzoz emisyonlarının azaltılması hedeflenmiştir.

- c) Sürdürülebilirlik:** Sistem modüler bir yapıda geliştirildiği için ileride yeni kavşaklar veya farklı sensörler eklenerek geliştirilebilir.
- d) Üretilirlik:** Projede kullanılan donanım ve yazılım bileşenleri kolay ulaşılabilir ve tekrar üretilir yapıdadır.
- e) Etik:** Sistem herhangi bir kişisel veri toplamamaktadır. Kamera görüntüleri yalnızca araç yoğunluğunu belirlemek amacıyla kullanılmaktadır.
- f) Sağlık:** Trafik yoğunluğunun azaltılması sürücülerin stres seviyesinin düşmesine ve daha güvenli bir sürüş ortamı oluşmasına katkı sağlayabilir.
- g) Güvenlik:** Sistemde olası hata durumları için manuel çalışma modu bulunmaktadır. Böylece görüntü işleme sistemi çalışmasa bile trafik ışıkları güvenli şekilde çalışmaya devam edebilmektedir.
- h) Sosyal ve politik sorunlar :** Proje, şehir içi ulaşımın daha verimli hale gelmesine katkı sağlamayı amaçlamaktadır. Trafik akışının iyileştirilmesi tüm sürücülere eşit şekilde fayda sağlamaktadır.

Tezin Adı:	GÖRÜNTÜ İŞLEME DESTEKLİ OTONOM KAVŞAK VE TRAFİK YÖNETİM SİSTEMİ
Projedeki Öğrenci Adları	SEDAT EROĞLU - BERK ALTINÖZ

EK-3 İş-Zaman Çizelgesi

İŞ-ZAMAN ÇİZELGESİ																																										
İş No	İşin Adı / Tanımı	Kim(ler) Tarafından Yapılacağı	AYLAR																																							
			Eylül				Ekim				Kasım				Aralık				Ocak				Şubat				Mart				Nisan				Mayıs				Haziran			
			1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4				
1	Literatür Araştırması ve	Sedat Eroğlu - Berk Altınöz																																								
2	Sistem Tasarımı ve Planlama	Sedat Eroğlu - Berk Altınöz																																								
3	Görüntü İşleme	Sedat Eroğlu																																								
4	Arduino Devresi ve Trafik Lambası	Berk Altınöz																																								
5	Python-Arduino Haberleşmesi	Berk Altınöz																																								
6	React Tabanlı Arayüz Geliştirme	Sedat Eroğlu																																								
7	Sistem Testleri ve Hata Giderme	Sedat Eroğlu - Berk Altınöz																																								
8	Tez Yazımı ve Dokümantasyon	Sedat Eroğlu - Berk Altınöz																																								

ÖZGEÇMİŞ

Kimlik Bilgileri

Adı Soyadı: SEDAT EROĞLU

Doğum Yeri: Hatay/Reyhanlı

Doğum Yılı: 13.01.2001

Haberleşme Bilgileri

Adres: Yeni mah. Mehmet Akif Ersoy caddesi No: 27 Kat: 2

Daire: 3 Hatay/Reyhanlı

Telefon: +90 534 621 34 43

E-posta: seroglu076@gmail.com

Bilimsel Çalışmalar

ÖZGEÇMİŞ

Kimlik Bilgileri

Adı Soyadı: BERK ALTINÖZ

Doğum Yeri: TEKİRDAĞ/MALKARA

Doğum Yılı: 17.06.2002

Haberleşme Bilgileri

Adres: Gazibey Mah. Eker Sitesi. B blok Daire:4 TEKİRDAĞ/MALKARA

Telefon: +90 535 271 84 57

E-posta: altnzberk@gmail.com

Bilimsel Çalışmalar